

Fonctionnement interne — agents-risques (projet E21)

Document **technique** : comment l'outil fonctionne de l'intérieur — architecture, flux de données, contrats JSON, mécanismes de sécurité, et correspondance avec les garde-fous du sujet et les recommandations OWASP LLM. Pour savoir quoi faire, voir `GUIDE_UTILISATION.md`.

Table des matières

- 1. Vue d'ensemble
- 2. Correspondance avec la démarche du cours
- 3. Architecture des composants
- 4. Contrat d'échange : l'enveloppe JSON
- 5. Séquence détaillée d'une analyse
- 6. Garde-fous de sécurité (G1→G10)
- 7. La matrice déterministe (G7)
- 8. Reprises et arrêt bavard (G8)
- 9. Recherche de connaissances (RAG) et base de connaissances (G10)
- 10. Le snapshot CVE
- 11. Journalisation (G5)
- 12. Simulateur déterministe vs fournisseur réel
- 13. Modèle de données
- 14. Validation par agent : tableau de référence
- 15. Limites et points d'attention
- 16. Glossaire

1. Vue d'ensemble



[illegible]

2. Correspondance avec la démarche du cours

Le cours (ch. 05 et p. 109) propose une chaîne : *décrire* → *identifier* (STRIDE/ LINDDUN) → *détailler* (*attaques*, ATT&CK) → *prioriser* (DREAD/CVSS) → *traiter*. Le système la reproduit en 6 étapes :

Étape	Composant	Cours
1. Décrire le système (DFD, frontières)	donnée statique (cases/ <i>*.md</i>)	ch. 05-06
2. Inventorier les actifs	AG1 Inventaire	ch. 05
3. Choisir le modèle de menaces	AG2 Modèle (STRIDE + LINDDUN)	ch. 06, 10
4. Identifier les menaces (actifs × frontières)	AG3 Menaces	ch. 06, 10
5. Évaluer (probabilité × impact)	AG4 Évaluation (matrice)	ch. 05, p. 8 du sujet
6. Traiter (réduire/transférer/éviter/accepter)	AG5 Traitement	ch. 05
7. Valider humainement et publier	Orchestrateur + CLI	condition du sujet

3.1 `src/core/` — le socle (aucune IA)

3.1 `src/core/` — le socle (aucune IA)

Fichier	Rôle
config.py	constantes : échelles, matrice , traitements autorisés/interdits, tentatives max, seuils
models.py	dataclasses Actif, Menace, Evaluation, Risque (avec valide_par: str \ None)
schemas.py	les schémas JSON des échanges (enveloppe + sortie par agent)
validation.py	validations : schéma + règles métier (sources, valide_par, traitements, échelles, matrice, unicité)
sanitizer.py	filtrage anti-injection de prompt : détection, neutralisation, balisage « donnée »
workspace.py	mémoire partagée de la session (produits + statuts + registre) et persistance workspace.json

Fichier	Rôle
<code>journal.py</code>	journalisation horodatée append-only (<code>journal.jsonl</code>)

3.2 `src/tools/` — les outils (lecture seule, rarement appelés par l'IA)

Outil	Fonction	Gas
<code>lire_fichier</code>	lit un descriptif de système uniquement dans <code>cases/</code> (taille max, pas de <code>...</code> , pas d'écriture)	G2
<code>chercher_connaissance</code>	recherche RAG dans <code>knowledge/*.md</code> et renvoie extraits + références	G4, G10
<code>calculer_niveau</code>	applique la matrice $P \times I \rightarrow$ <code>niveau</code> (déterministe)	G7
<code>rechercher_cve</code>	interroge le snapshot local CVE/CVSS	G4

3.3 `src/llm/` — la couche modèle (remplaçable, G9)

Fichier	Rôle
<code>interface.py</code>	contrat <code>FournisseurLLM</code> (Protocol) + <code>extraire_json</code>
<code>dummy.py</code>	simulateur déterministe : sert les sorties de l'analyse de référence (démon + tests)
<code>openai_compat.py</code>	fournisseur réel : <code>POST {base_url}/chat/completions</code> (urllib standard, sans dépendance)

3.4 `src/agents/` — l'orchestration et les consignes

Fichier	Rôle
<code>base.py</code>	classe <code>Agent</code> : assemblage du prompt, sandbox d'outils, point d'appel unique
<code>prompts.py</code>	les 5 consignes (AG1→AG5), le bloc système commun, listes d'outils et requêtes RAG
<code>agents.py</code>	les 5 classes concrètes + ordre d'exécution
<code>orchestrator.py</code>	pipeline (AG1→AG5), validation, reprises , normalisation déterministe, human loop, outils

3.5 `src/cli/` — le point d'entrée

Fichier	Rôle
<code>main.py</code>	argparse : sous-commandes <code>run</code> et <code>test-injection</code> , session horodatée, écriture des 3 livrables

4. Contrat d'échange : l'enveloppe JSON

Toutes les sorties d'agents obéissent à **la même enveloppe** (`core/schemas.py`) :

```
{
  "agent": "AG3",
  "resume": "...",
  "sortie": { "..."},
  "sources": ["STRIDE", "01_Cadrage (frontières)",
  "incertitudes": []
}
```

- `sources` : **obligatoire, non vide** (G4 — parade anti-hallucination).
- `incertitudes` : facultatif — l'agent y signale ce qu'il ne sait pas.
- `valide_par` : **n'apparaît pas** dans cet échange, et quand il apparaît dans un risque il doit être `null` (G6). Le schéma AG5 impose même `"type": "null"`.

Les schémas de `sortie` par agent (cf. `SCHEMAS_SORTIE`) :

Agent	<code>sortie</code> contient	Contraintes notables
AG1	<code>actifs[]</code>	<code>id = A-\d{2}</code> , type parmi 6 valeurs, <code>criticite</code> ∈ {C,I,A}, valeur parmi 4
AG2	<code>modele_retenu</code> , <code>modeles_complementaires</code>	Échelles normalisées (aucune échelle d'évaluation sans aucun doublon)
AG3	<code>menaces[]</code>	<code>id_menace = M-\d{2}</code> , <code>frontiere = "1"..."6"</code> , <code>categorie = STRIDE-... ou LINDDUN-...</code>
AG4	<code>evaluations[]</code>	<code>id_menace = M-\d{2}</code> , P/I dans les échelles de configuration
AG5	<code>risques[]</code>	<code>id = R-\d{2}</code> , <code>menace_id</code> requis , <code>traitement</code> ∈ 4 réponses, <code>valide_par</code> <code>null</code>

`additionalProperties: true` permet aux agents d'enrichir (ex. `menace_id`, `contraintes`), sans jamais supprimer les champs requis.

5. Séquence détaillée d'une analyse

- Entrée** (CLI `commande_run`) : 1. `lire_fichier("cases/casB_mediconsult.md")` — lecture seule, chemin restreint, ≤ 100 000 octets.
- Phase 1 — assainissement** (`Orchestrateur.analyser`) : 2. `sanctionner_document(description)` : a. détection de motifs d'injection (regex FR/EN), b. neutralisation (`[[DONNEE-NON-EXECUTABLE]]`), c. balisage en bloc **donnée** : `<<<DONNEES-SYSTEME-A-ANALYSER>>> ... <<<FIN-DONNEES>>>`. 3. si détections → événement `injection_detectee` dans le journal. 4. le `contexte_metier` (si fourni) subit le même traitement (G1) avant injection.
- Phase 2 — boucle agent par agent** (ordre AG1→AG5) :

Pour chaque agent A : 5. `_preparer_rag(A)` : les requêtes RAG de `REQUETES_RAG[A]` sont exécutées sur la base de connaissances ; les extraits (dédoublonnés, ≤ 6) sont injectés dans le prompt. 6. si `A == AG3` : `_preparer_cve_ag3()` complète le contexte avec le snapshot CVE. 7. `agent.definir_contexte(contexte)` : l'agent reçoit les sorties normalisées des étapes précédentes (actifs, modèle, menaces, évaluations). 8. **Le prompt est assemblé** (`base.Agent.construire_prompt`) : marqueur + consigne + outils + connaissances(RAG) + DONNEES + format attendu + rappels de sécurité. 9. `agent.produire()` → un appel LLM (`llm.complete(système, utilisateur)`). 10. le brut est parsé en JSON (`json.loads`), sinon `echec_json` → reprise (voir §8). 11. `toutes_les_erreurs(...)` : pile de validation (schéma, sources, `valide_par`, règles métier propres à l'agent). 12. si `A == AG4` et pas d'erreur : `_appliquer_niveaux_deterministes` recalcule chaque niveau via la matrice — **le LLM ne décide jamais du niveau** (G7). 13. si aucune erreur → statut ok, la sortie est mémorisée dans le workspace (`set_etape`) et **normalisée** pour l'étape suivante (`_injecter_etape`). 14. sinon → `echec_schema`, les erreurs sont réinjectées comme `erreurs_reparation` dans le prompt du prochain essai (retour de correction), jusqu'à 3 tentatives. 15. au-delà de 3 tentatives → **EtapeImpossibleError** : arrêt explicite (G8), aucun fichier de résultat n'est publié, le CLI sort en code 1.

Phase 3 — validation humaine (`valider_humainement`) : 16. `registre_provisoire()` : copie profonde des risques d'AG5 (le produit d'AG5 n'est **jamais muté**), avec `valide_par = null`. 17. pour chaque risque : question interactive a/d/c. 18. chaque décision → événement `validation_humaine` (accepte / diffère / corrige).

Phase 4 — persistance : 19. `workspace.set_final(registre)` puis `workspace.sauvegarder()` → `workspace.json`. 20. `registre_final.json` est écrit ; événement `session_fin` horodaté.

6. Garde-fous de sécurité (G1→G10)

Correspondance exacte avec le référentiel du projet (p. 20 du sujet) et OWASP LLM :

#	Garde-fou	Mécanisme réel	Code	OWASP
G1	Filtrage des entrées (injection)	regex FR/EN + neutralisation + séparation consigne/donnée (bloc <<<DONNEES>>>) + <code>contexte_metier</code> assaini	<code>core/sanitizer.py</code> , <code>agent/orchestrator.py</code> (étapes 2-4)	LLM01
G2	Outils lecture seule	registre d'outils + <code>SandboxOutils</code> (liste blanche par agent, <code>PermissionError</code> sinon) ; <code>lire_fichier</code> restreint à <code>cases/</code>	<code>tools/registry.py</code> , <code>tools/file_reader.py</code>	LLM06
G3	Anonymisation	cas d'étude fictif ; avertissement explicite pour le mode réel ; zéro donnée réelle dans le pipeline	<code>README.md</code> , prompts	LLM02

#	Garde-fou	Mécanisme réel	Code	OWASP
G4	Sources citées	sources requis non vide par le schéma (enveloppe) et par risque AG5 ; RAG renvoie reference à recopier	core/validation.py, LLM07/tools/knowledge.py	LLM07 (anti-poisoning)
G5	Journalisation	journal JSON-lines horodaté de chaque appel, chaque outil, chaque décision	core/journal.py	—
G6	Validation humaine	valide_par écrit uniquement par validation AG5 impose "type": "null" ; la chaîne "null" est rejetée	core/validation.py, core/schemas.py, agents/orchestrator.py	—
G7	Niveau déterministe	matrice P×I appliquée par l'orchestrateur (AG4) et re-vérifiée sur chaque risque final (AG5) ; aucune « auto-autorisation » d'échelle inventée	tools/matrice.py, core/validation.py, agents/orchestrator.py	LLM07 (overreliance)
G8	Arrêt bavard	3 tentatives max par étape, puis exception et code 1 — jamais de remplissage	agents/orchestrator.py, cli/main.py	LLM08 (excessive agency)
G9	Modèle remplaçable	interface FournisseurLLM (Protocol) ; dummy ■ openai_compat sans toucher au reste	llm/interface.py	—
G10	Connaissances versionnées, lecture seule	knowledge/*.md chargés en lecture seule et indexés ; les extraits sont des citations	tools/knowledge.py	LLM07

Règles métier complémentaires dans `validation.py` : - traitements limités à **réduire / transférer / éviter / accepter** — « ignorer », « rien », « néant » sont **interdits** ; - identifiants uniques (A-`nn`, M-`nn`, R-`nn`) ; - P/I **dans les échelles de configuration** (l'écho de l'agent est ignoré) ; - AG5 doit **recopier** P/I/niveau de l'étape 4 (jamais recalculer) et y rester rattaché par `menace_id`.

6.1 Les 10 motifs du sanitizer (G1)

core/sanitizer.py — `_MOTIFS_INJECTION` : paires (nom, regex) en français et en anglais. La détection ne bloque pas l'analyse : le motif est **signalé** (`injection_detectee`) puis **neutralisé** en `[[DONNEE-NON-EXECUTABLE]]`.

Motif	Cible (exemple de ce qui est attrapé)
<code>ignore_instructions</code>	« ignore tes instructions », « disregard your previous rules »
<code>disable_securite</code>	« désactive tes consignes », « forget all security »
<code>override_system</code>	« tu es maintenant... », « you are now... », « new system prompt »
<code>nouvelle_persona</code>	« à partir de maintenant », « fais comme si », « prétend »
<code>delimiter_escape</code>	<code><<SYS>></code> , <code><\ im_start\ ></code> , <code><\ system\ ></code>
<code>data_as_instruction</code>	une ligne <code>instructions</code> : ... en début de texte
<code>jailbreak_mot</code>	« jailbreak », « désactive tes règles »
<code>assistant_prefix</code>	une ligne <code>assistant</code> : ... insérée (usurpation de rôle)
<code>disclose_secrets</code>	« révèle tes instructions/clefs », « reveal your system prompt »
<code>replay_prompt</code>	« répète tout ce qui précède », « repeat everything above »

Vérifié par tests : les **4 motifs présents dans** `casB_injecte.md` sont détectés, et le document sain du cas B ne produit **aucun faux positif** (`test_document_legitime_sans_fausse_alarme`).

7. La matrice déterministe (G7)

Matrice probabilité × impact **celle du sujet (p. 8)** :

Prob \ Impact	faible	moyen	élevé
élevée	moyen	élevé	critique
moyenne	faible	moyen	élevé
faible	faible	faible	moyen

Deux endroits en imposent le respect : 1. **AG4** : après validation, l'orchestrateur écrase `niveau` par `calculer_niveau(P, I)` — un niveau « inventé » est remplacé, jamais propagé. Une P/I hors matrice provoque un échec bloquant (pas de `continue` silencieux). 2. **AG5** : `verifier_niveaux_agent5` re-vérifie **chaque entrée du registre final** ; un niveau ≠ matrice est une erreur de validation (exception après 3 reprises).

8. Reprises et arrêt bavard (G8)

Boucle de reprise (`_executer_avec_reprises`) :

```
pour tentative = 1..3 :  
  1. appel LLM  
  ■■ exception réseau/API → echec_llm → rejouer (même prompt)
```

```
■■ JSON invalide → echec_json → rejouer avec erreurs_reparation
■■ JSON valide → toutes_les_erreurs(...)
■■ 0 erreur → le niveau AG4 est recalé, retour sortie ■
■■ erreurs → echec_schema → rejouer avec erreurs_reparation
après 3 tentatives → EtapeImpossibleError (code 1, aucun livrable publié)
```

La **réparation** consiste à réinjecter les erreurs dans le prompt du tour suivant : l'agent sait *pourquoi* sa sortie a été refusée. Les messages d'erreur sont **tronqués à 160 caractères** avant journalisation (pas de fuite de contenu LLM).

9. Recherche de connaissances (RAG) et base de connaissances (G10)

RAG minimaliste, sans dépendance vectorielle :

1. `knowledge/` contient 5 documents Markdown : `stride.md`, `linddun.md`, `iso27002.md`, `anssi.md`, `attck.md`.
2. À chaque démarrage, `BaseConnaissances` les charge et les **découpe en blocs** (par titre `#`).
3. `chercher_connaissance(requête)` : mots-clés (≥ 3 lettres, mots vides exclus) → score de cooccurrence par bloc → les **top extraits** sont renvoyés avec leur **référence** (`knowledge/stride.md` · STRIDE – grille d'identification).
4. L'orchestrateur pré-extraie **par agent** (requêtes fixes de `REQUETES_RAG`) et injecte ≤ 6 extraits dans le prompt.
5. L'agent est invité à **recopier ces références dans sources** — d'où la vérification G4.

C'est une forme de *grounding* : le modèle répond à partir d'un corpus contrôlé et citable, au lieu de puiser dans sa seule mémoire.

10. Le snapshot CVE

`data/cve_snapshot.json` : 5 CVE réelles avec score CVSS et référence NVD.

- `rechercher_cve(mots=..., version=...)` filtre par mots-clés ; **sans critère**, réponse vide.
- Utilisé pour **AG3** : l'orchestrateur pré-charge les CVE proches des technologies du cas (`webtrc/tls/api`) dans le contexte de l'agent.
- ■■ C'est un **instantané de démonstration**, figé : la fonction renvoie en toutes lettres qu'il ne faut pas inventer d'autres et qu'il faut consulter le **NVD** en production. Cette limitation est volontaire et documentée (anti-hallucination).

11. Journalisation (G5)

`core/journal.py` — écriture **append-only** en JSON-lines, horodatage UTC ISO-8601.

Événements émis :

Événement	champs	émis par
<code>session_debut</code> / <code>session_fin</code>	chemin / statut, nb_risques	CLI, orchestrateur

Événement	champs	émis par
session_fin_motif	motif (arrêt bavard)	CLI (erreur G8)
appel_agent	agent, tentative, statut (ok / echec_schema / echec_json / echec_llm / outil), outils, octets entrée/sortie, erreurs	orchestrateur, sandbox
injection_detectee	liste {motif, extrait}	orchestrateur (G1)
rag_echec	agent, erreur	orchestrateur
validation_humaine	risque, valide_par, décision	orchestrateur

12. Simulateur déterministe vs fournisseur réel

FournisseurSimule (dummy.py) - Sert les sorties figées qui correspondent à **l'analyse manuelle de référence** (14 actifs, STRIDE+LINDDUN, 10 menaces, 10 évaluations, 10 risques). - Routage par **marqueur** dans le prompt (__AGENT_1_INVENTAIRE__ ...). - Un mode `troubler=True` (tests uniquement) produit des sorties **volontairement invalides** (valide_par rempli, sources vides, traitement « ignorer », niveau faux) pour prouver que les garde-fous bloquent. - Usage : démo sans clé, tests déterministes, démonstration « 10/10 » vs référence.

OpenAICompatProvider (openai_compat.py) - `POST {base_url}/chat/completions` avec `messages=[system, user]`, `temperature=0.2`. - Clé et modèle **uniquement** via variables d'environnement (aucun secret en dur). - `extraire_json` (interface.py) : si la réponse est bavarde, extrait le premier `{...}` au dernier `}` ; sinon `ValueError` → reprise. - ■■ Ne jamais envoyer de **données réelles** (G3).

Remplaçabilité (G9) : tout objet implémentant `complete(système, utilisateur) → str` (contract `FournisseurLLM`) peut être branché sans modifier le pipeline.

13. Modèle de données

Dataclasses (core/models.py) — miroir exact des schémas JSON :

- **Actif** : id, nom, type, description, criticite, valeur, propriétaire, contraintes
- **Menace** : id_menace, actif, frontiere, description, categorie, vulnerabilite, source_menace
- **Evaluation** : id_menace, probabilite, impact, niveau, justification
- **Risque** : id, actif, menace_id, menace, categorie, probabilite, impact, niveau, traitement, mesures, justification, sources, risque_residuel, propriétaire, valide_par

L'**enveloppe** (agent, resume, sortie, sources, incertitudes) est le seul format qu'un agent peut rendre ; le workspace stocke ces enveloppes **validées et normalisées**.

14. Validation par agent : tableau de référence

Récapitulatif des vérifications appliquées à chaque étape (toutes_les_erreurs) :

Étape	Schéma	Sources ≥ 1	valide_par = null	Unicité ids	Autre règle
AG1	■	■	–	id actifs	type, criticité, valeur c les énumérations (via schéma)
AG2	■	■	–	–	échelles normalisées sans doublons
AG3	■	■	–	id_menace	frontiere ∈ 1..6, categorie ∈ STRIDE/LINDDUN (via schéma)
AG4	■	■	–	id_menace	P/I dans les échelles de configuration ; niveau recalculé (pas d'écho)
AG5	■	■	■	id	menace_id requis + connu ; traitement autorisé (« ignorer » interdit) ; P/I/niveau = étape 4 ; niveau = matrice

15. Limites et points d'attention

1. **Sanitizer à base de motifs** : il détecte les injections *connues* (10 motifs FR/EN, § 6.1). Il ne prétend pas à l'exhaustivité — la **séparation consigne/donnée** et la **validation stricte des sorties** sont les couches réellement décisives (défense en profondeur, à l'image du cours).
2. **CVE figées** (`cve_snapshot.json`) : ce sont des données de démo ; en production il faut interroger le NVD. La fonction refuse d'ailleurs d'en inventer.
3. `MAX_PROMPT_OCTETS` : constante définie dans `config.py` mais **pas encore exploitée** (le prompt est borné indirectement par `MAX_DOCUMENT_OCTETS`).
4. **RAG simple** : recherche par mots-clés, pas de plongements vectoriels — c'est un choix pédagogique (aucune dépendance, reproductibilité totale).
5. **Échelles normalisées** : AG2 choisit ses échelles « pour la vitrine » mais la validation utilise les échelles de `config.py` — volontaire : les 3 valeurs du sujet sont la référence commune.
6. **Dry-run vs réel** : le simulateur **reproduit la référence manuelle** ; seul le mode réel fait varier les réponses. C'est la base de la comparaison du jalon 4.

16. Glossaire

Terme	Définition
Agent	composant IA qui produit une étape de l'analyse (un prompt + outils + contrat de sortie)
Orchestrateur	coordonne les agents : contexte, validation, reprises, mémoire, journal
Enveloppe	contrat JSON commun de sortie (agent , resume , sortie , sources , incertitudes)
RAG	<i>Retrieval-Augmented Generation</i> : injection d'extraits d'une base de connaissance dans le prompt
Sanitizer	filtre d'entrée : détecte/neutralise les tentatives d'injection de prompt
Matrice P×I	outil déterministe « probabilité × impact → niveau » (le LLM n'en décide pas)
Arrêt bavard	en cas de rejet répété (3×), l'outil s'arrête en erreur au lieu d'inventer
valide_par	champ signé par l'humain uniquement ; null tant que rien n'est validé
Sémantique des traitements	réduire / transférer / éviter / accepter ; « ignorer » toujours interdit
Snapshot CVE	jeux de données CVE/CVSS local et figé, à rafraîchir via le NVD en production

Références croisées : [README.md](#) (vue raccourcie), [GUIDE_UTILISATION.md](#) (quoi faire), [../02_Architecture_MultiAgents.md](#) (conception), [../03_Analyse_Manuelle_Reference.md](#) (référence manuelle), [../04_Tests_Comparaison_Agents.md](#) (résultats de tests), [../07_Guide_Repetition_Soutenance.md](#) et [../08_Demo_Scriptee.md](#) (préparation soutenance), [../utils/md_to_pdf.py](#) (génération des PDF des livrables).